# Taller guiado: observabilidad con Spring Boot, Prometheus, Loki y Grafana

**Duración:** 20 minutos · **Qué necesitas:** un navegador y una cuenta de GitHub (gratis). No hay que instalar nada.

Vas a levantar una mini tienda en Spring Boot que tiene **tres problemas escondidos**. Tu misión es encontrarlos usando solo la telemetría: métricas en Prometheus, logs en Loki y el dashboard de Grafana.

| Tiempo | Paso |
|---|---|
| 0-4 min | 1. Levantar el entorno |
| 4-6 min | 2. Generar tráfico |
| 6-16 min | 3. Tres misiones |
| 16-20 min | 4. Reto: crear tu propia alerta |

---

## 1. Levantar el entorno (0-4 min)

### Opción A: GitHub Codespaces (recomendada, nada que instalar)

1. Abre este enlace con tu cuenta de GitHub: **https://codespaces.new/JuanTobon7/arquitectura-software/tree/claude/project-thread-ga3dib?devcontainer_path=.devcontainer/observabilidad/devcontainer.json**
2. Pulsa **Create codespace** y espera a que cargue VS Code en el navegador (1-2 min la primera vez).
3. En la terminal de abajo (ya está en la carpeta `observabilidad-demo`) ejecuta:

   ```
   docker compose up -d --build
   ```

   La primera vez compila la API con Maven dentro de Docker (unos 2 minutos).
4. Verifica que la API respondió:

   ```
   curl -s localhost:8080/actuator/health
   ```

   Debe decir `{"status":"UP"}`. Si dice *connection refused*, espera 20 segundos y repite.
5. Abre la pestaña **PORTS** (junto a la terminal) y haz clic en el globo del puerto **3000 (Grafana)**. No pide usuario ni contraseña.

### Opción B: Docker Desktop en tu computador

```
git clone -b claude/project-thread-ga3dib https://github.com/JuanTobon7/arquitectura-software.git
cd arquitectura-software/observabilidad-demo
docker compose up -d --build
```

Grafana queda en http://localhost:3000 y Prometheus en http://localhost:9090.

> **¿Te atrasaste?** Trabaja con el compañero de al lado. No hace falta que todos tengan el entorno para resolver las misiones.

---

## 2. Generar tráfico (4-6 min)

En la terminal:

```
DURACION=6m docker compose --profile carga run -d --rm k6
```

Esto lanza k6 en segundo plano: unas 40 peticiones por segundo durante 6 minutos contra cuatro endpoints:

| Endpoint | Qué hace |
|---|---|
| `POST /api/pagos` | Llama a una "pasarela de pagos" externa |
| `GET /api/pedidos/n-mas-1` | Lista 100 pedidos con sus ítems |
| `GET /api/pedidos/optimizado` | Lista los mismos 100 pedidos |
| `GET /api/catalogo` | Devuelve el catálogo |

En Grafana abre el dashboard **API Demo - Observabilidad** (es la página de inicio). Espera 1 minuto a que se llenen las gráficas.

---

## 3. Tres misiones (6-16 min)

Para escribir consultas usa **Grafana** → **Explore** (icono de brújula en el menú izquierdo, o `/explore` en la URL). Arriba a la izquierda eliges la fuente de datos: **Prometheus** para métricas (PromQL) o **Loki** para logs (LogQL).

### Misión 1: ¿el promedio miente? (PromQL)

Elige **Prometheus**, cambia el editor a modo **Code** y ejecuta estas tres consultas, una a la vez:

```
# Latencia promedio de /api/pagos (en segundos)
sum(rate(http_server_requests_seconds_sum{uri="/api/pagos"}[1m]))
  / sum(rate(http_server_requests_seconds_count{uri="/api/pagos"}[1m]))
```

```
# Percentil 95
histogram_quantile(0.95, sum by (le)
(rate(http_server_requests_seconds_bucket{uri="/api/pagos"}[1m])))
```

```
# Percentil 99
histogram_quantile(0.99, sum by (le)
(rate(http_server_requests_seconds_bucket{uri="/api/pagos"}[1m])))
```

**Preguntas:**

- ¿Cuánto da el promedio? ¿Y el p99? ¿Cuántas veces más grande es el p99?
- Si tu SLO fuera "el 99% de los pagos en menos de 500 ms", ¿se cumple?
- Revisa el panel *Hilos Tomcat ocupados* y *CPU del proceso* del dashboard. ¿El servidor está saturado? Entonces, ¿dónde está el problema?

### Misión 2: cazar el N+1 (métrica + LogQL)

Los dos endpoints de pedidos devuelven **exactamente el mismo JSON**. Compruébalo:

```
curl -s localhost:8080/api/pedidos/n-mas-1 | head -c 150; echo
curl -s localhost:8080/api/pedidos/optimizado | head -c 150; echo
```

1. En el dashboard mira el panel **Latencia p95 por endpoint**. ¿Cuál de los dos es más lento y cuánto?
2. Mira el panel **Consultas SQL por request**. ¿Cuántas consultas hace cada uno? ¿Por qué ese número exacto, si hay 100 pedidos?
3. Ahora busca la evidencia en los logs. En Explore elige **Loki** y ejecuta:

   ```
   {app="api-demo"} | json | mdc_sql_consultas > 50
   ```

   Abre una línea. ¿Qué campos trae? Copia un `mdc_request_id`.
4. Abre `api/src/main/java/co/edu/demo/observabilidad/PedidoRepository.java` y `PedidoController.java`. ¿Qué diferencia hay entre `findAll()` y `findAllConItems()`?

### Misión 3: de la alerta al log

1. Abre Prometheus (puerto **9090** → menú **Alerts**). ¿Qué alerta está en estado **FIRING** y para qué endpoint?
2. En el dashboard, panel **Tasa de errores 5xx**: ¿qué porcentaje falla? ¿Hay errores en otros endpoints?
3. En Explore con **Loki**:

   ```
   {app="api-demo", level="ERROR"}
   ```

   ¿Qué dice el mensaje? ¿El error es de nuestra API o de una dependencia?
4. Cuenta errores por minuto, solo con logs:

```
sum(count_over_time({app="api-demo", level="ERROR"}[1m]))
```

---

## 4. Reto: tu propia alerta (16-20 min)

La alerta de latencia que trae el proyecto usa el **p95 > 500 ms** y **no se dispara**, aunque hay pagos de casi 2 segundos. Crea una alerta sobre el **p99**:

1. Abre `infra/prometheus/alertas.yml` y agrega al final de `rules:` (respeta la indentación de las otras reglas):

```
      - alert: LatenciaP99Alta
        expr: |
          histogram_quantile(0.99,
            sum by (le, uri)
(rate(http_server_requests_seconds_bucket{uri!~"/actuator.*"}[2m]))
          ) > 1
        for: 30s
        labels:
          severity: warning
        annotations:
          summary: "p99 de {{ $labels.uri }} por encima de 1 s"
```

2. Recarga la configuración de Prometheus sin reiniciarlo:

```
curl -X POST localhost:9090/-/reload
```

3. En Prometheus → **Alerts** deberías ver `LatenciaP99Alta` pasar de *Pending* a **Firing** en menos de un minuto.

**Bonus:** en Grafana edita el panel *Latencia p95 por endpoint* y agrega una segunda consulta con el p99. ¿Qué endpoint cambia más?

---

## Para comparar: resultados de referencia

Una corrida de 4 minutos (9.603 requests) dio estos valores. Los tuyos variarán porque la latencia es aleatoria, pero deben quedar en el mismo orden de magnitud.

| Métrica | /api/pagos | /api/pedidos/n-mas-1 | /api/pedidos/optimizado |
|---|---|---|---|
| Promedio | 103 ms | 218 ms | 3 ms |
| p95 | 372 ms | 224 ms | 5 ms |
| p99 | 1.574 ms | 244 ms | 7 ms |
| Errores 5xx | 4,4 % | 0 % | 0 % |
| Consultas SQL por request | 0 | 101 | 1 |

### Respuestas

- **Misión 1:** el promedio (~~100 ms) oculta la cola: el p99 (~~1,5 s) es unas 15 veces mayor. El SLO de 500 ms en p99 no se cumple. Tomcat usa como máximo unos 5 de 50 hilos y la CPU está por debajo del 6%: el servidor no está saturado. El problema es la dependencia externa (la pasarela), así que escalar servidores no lo arregla; sí ayudan los timeouts y un circuit breaker.
- **Misión 2:** `n-mas-1` hace 101 consultas (1 para los pedidos + 1 por cada uno de los 100 pedidos, porque la relación `items` es `LAZY`). `optimizado` hace 1 sola consulta con `JOIN FETCH` y es unas 45 veces más rápido.
- **Misión 3:** `TasaErroresAlta` para `/api/pagos`, con ~4% de errores. Los logs dicen "*La pasarela de pagos respondio timeout*": el fallo es de la dependencia, no de la base de datos ni de la red.
- **Reto:** la alerta de p99 sí se dispara para `/api/pagos`, mientras que la de p95 no. Elegir el percentil del SLO cambia lo que ves.

## Al terminar

```
docker compose down
```

Si usaste Codespaces, detén el codespace desde https://github.com/codespaces para no gastar tus horas gratis.